



Python, Machine Learning, AI... and Astronomy

Dr. Brian R. Kent, Director of Science Communications
National Radio Astronomy Observatory <https://www.cv.nrao.edu/~bkent/>
BlueSky, Twitter & Instagram: @VizAstro
YouTube: Visualize Astronomy June 2026

Overview

- History of Python
- Why use Python in Astronomy?
- Why Revision Control is important...
- Google Colab and simple AI examples
- Data Colorimetry and Presentation
- AI Prompt Examples
- Resources to explore further
- All the slides and notebooks will be shared
- ***Recommendations***



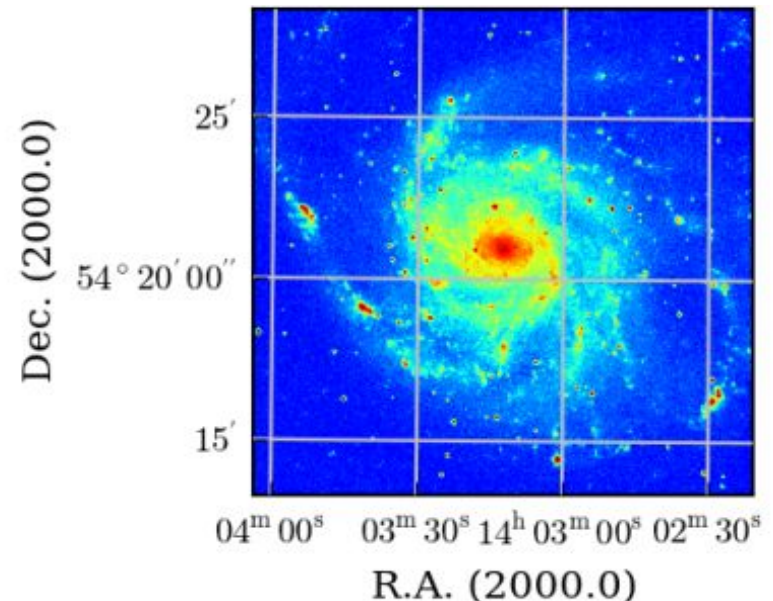
- High level programming language that is scriptable and supports many programming styles/paradigms.
- Created in the late 80s/early 90s by *Guido van Rossum*, named after *Monty Python*
- The **Python Standard Library** can be used to accomplish a wide variety of tasks - always check there first!

<https://ivastar.github.io/python-novice-astro/>

Jun 2026	Jun 2025	Change	Programming Language		Ratings	Change
1	1			Python	18.96%	-6.91%
2	3	▲		C	10.77%	+1.30%
3	2	▼		C++	8.03%	-2.65%
4	4			Java	7.90%	-0.94%
5	5			C#	4.85%	+0.17%
6	6			JavaScript	3.04%	-0.17%
7	8	▲		Visual Basic	2.80%	+0.59%
8	12	▲▲		SQL	1.77%	+0.23%
9	14	▲▲		R	1.69%	+0.30%
10	9	▼		Delphi/Object Pascal	1.54%	-0.60%
11	16	▲▲		Scratch	1.46%	+0.27%
12	18	▲▲		Rust	1.26%	+0.30%
13	7	▼▼		Go	1.20%	-1.08%
14	15	▲		PHP	1.17%	-0.08%
15	25	▲▲		Swift	1.00%	+0.27%
16	11	▼▼		Ada	0.97%	-0.74%
17	10	▼▼		Fortran	0.95%	-0.91%
18	13	▼▼		Perl	0.92%	-0.55%
19	19			Assembly language	0.87%	-0.04%
20	17	▼		MATLAB	0.85%	-0.28%

Python and Astronomy

- With iPython, it is used as the command line shell & interpreter for CASA
- Used for the framework for VLA/GBT imaging/ALMA pipelines
- Can interface with AIPS via Obit
- Soon to be used - Dysh for GBT
- Used in PRESTO - pulsar data reduction
- Many modules and libraries available - numpy, matplotlib, Kapteyn, astropy, Pandas, ApIPy, etc.
- Managed via pip or Anaconda
- Scriptable for astronomers
OOP for engineers
- Use in environments like Colab and github.dev



Popular Python Resources

iPython - more user friendly shell

Astropy - great for data import, manipulation, catalog queries

AplPy - general image/coordinate display utility

Matplotlib - general purpose plotting tool

Scipy - numpy and fitting routines (some overlap with astropy...)

Kapteyn - Good for mapping projections

AstroML - Machine Learning <http://www.astroml.org/index.html>

PANDAS - Data Analysis Library <http://pandas.pydata.org/>

Python Data Science Handbook:

<https://jakevdp.github.io/PythonDataScienceHandbook/>

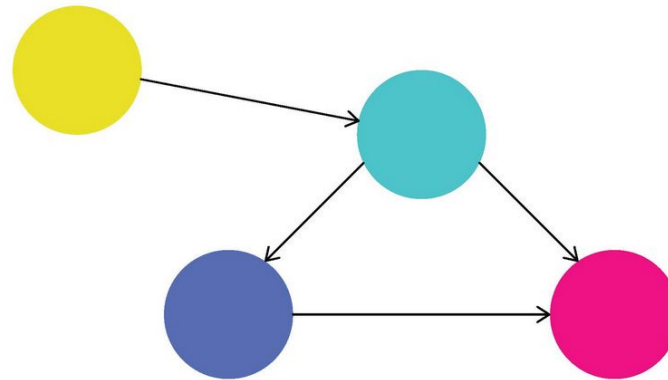
Recommendation: ~~Learn to use these inside and out!~~

Recommendation: Use AI Systems to leverage these libraries!

«packt»

Causal Inference and Discovery in Python

Unlock the secrets of modern causal machine learning
with DoWhy, EconML, PyTorch and more



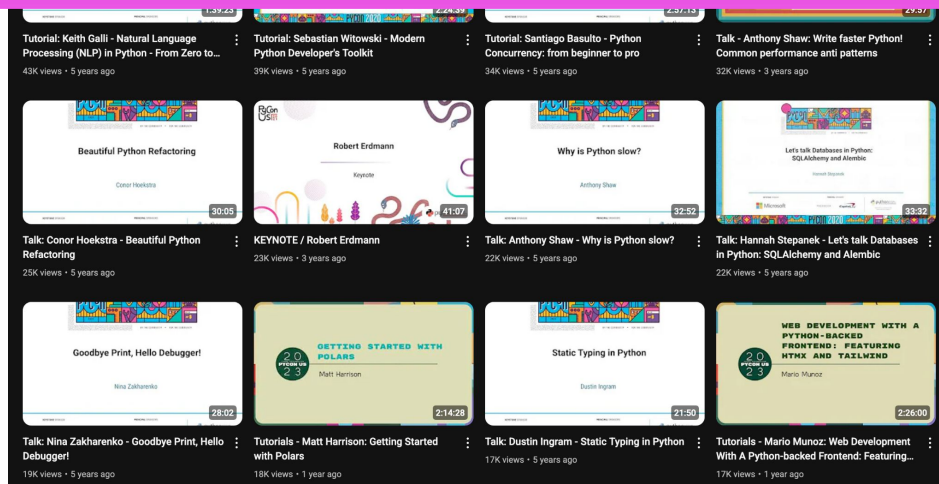
ALEKSANDER MOLAK

Foreword by Ajit Jaokar, Course Director: Artificial Intelligence at University of Oxford

<https://github.com/PacktPublishing/Causal-Inference-and-Discovery-in-Python/tree/main>

Where to start?

- <https://www.youtube.com/@PyConUS/videos>
- Pycon: <https://us.pycon.org/2026> (and video tutorials)



Revision Control

Revision Control for Software

Popular revision control systems (RCS) include SVN, CVS, Mercurial, and **git**.

Track changes made to files - good for collaboration among teams or to see when and where changes were made to your code!

Repo for this presentation:

<https://github.com/brkent/SummerStudent2026>

- | | |
|---------------------|------------------------|
| 1. Plotting | 6. FITS images |
| 2. 2D Plotting | 7. VLASS and subimages |
| 3. HR Diagram | 8. Image comparison |
| 4. Cosmology | 9. Contours |
| 5. Fitting Function | 10. Neural Networks |

git revision control

```
mkdir sumstudent
```

```
cd sumstudent
```

```
git clone https://github.com/brkent/SummerStudent2026.git
```

This will create and link your directory
to the git repository

git revision control

#Create a readme file

```
touch README.md
```

```
git add README.md
```

```
git commit -m "Initial README commit"
```

```
git push -u origin main
```

This is your modern scientific lab notebook!

Recommendation:

If you do not already know how to use git, please take some time this summer and practice with these tutorials:

<https://docs.github.com/en/get-started>

Google Colab



Colaboratory is a **Google** research project created to help disseminate machine learning education and research. It's a Jupyter notebook environment that requires no setup to use and runs entirely in the cloud. Now interfaces with Gemini...

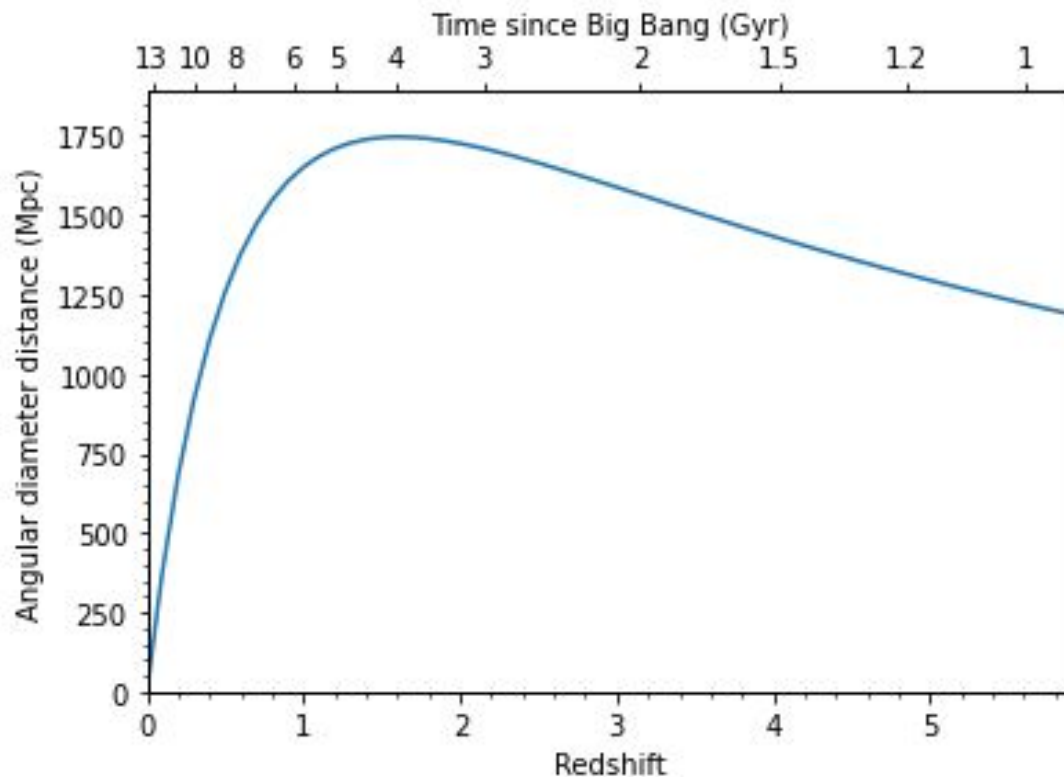
https://github.com/brkent/SummerStudent2026/blob/main/example05_gaussian_fit_seaborn.ipynb

Also works in github.dev...

Recommendation: Learn to prototype bit of code in the cloud and make **clean** Jupyter notebooks. Make your work portable and shareable! If you can point potential employers at your work and it is organized/documentated - all the better!

Example: Astropy Cosmology Calc

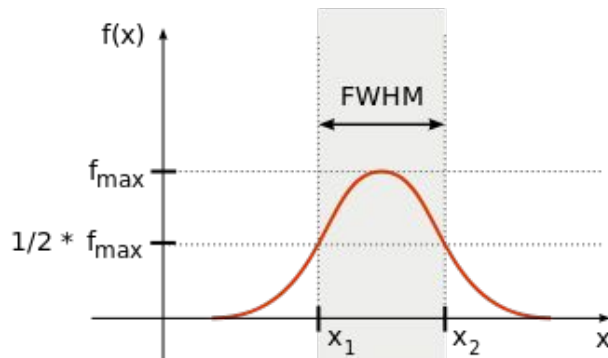
https://github.com/brkent/SummerStudent2024/blob/main/example03_cosmology.ipynb



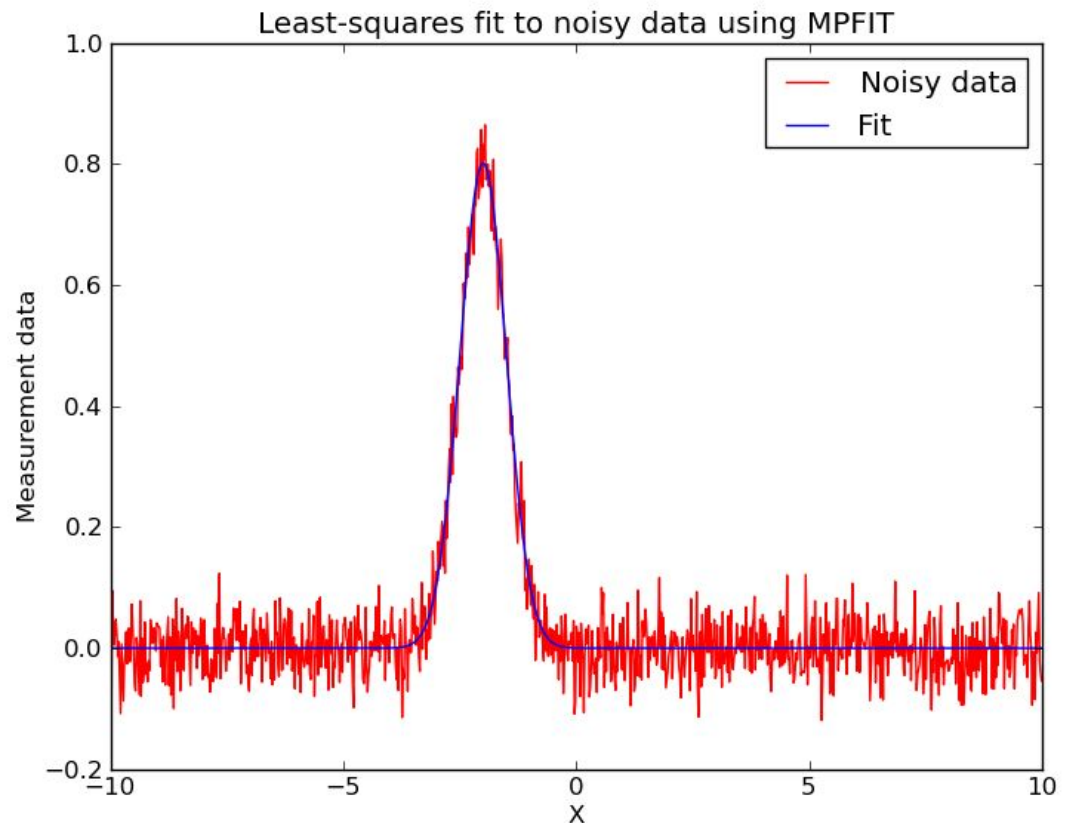
Example: Create a noisy spectrum and fit a Gaussian

https://github.com/brkent/SummerStudent2026/blob/main/example05_gaussian_fit_seaborn.ipynb

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-x_0)^2}{2\sigma^2}\right]$$



$$\text{FWHM} = 2\sqrt{2\ln 2} \sigma \approx 2.355 \sigma.$$



Another series of great fitting examples...

<https://learn.astropy.org/tutorials/Models-Quick-Fit.html>

“That cluttered slide with multiple tiny plots, and with NO units, is SO easy to read from the back of the room!”

- Never said by a meeting attendee in science

Colorimetry for Visualization

*Want your plots
to look top notch?*



seaborn

<https://seaborn.pydata.org/>

https://seaborn.pydata.org/tutorial/color_palettes.html#general-principles-for-using-color-in-plots

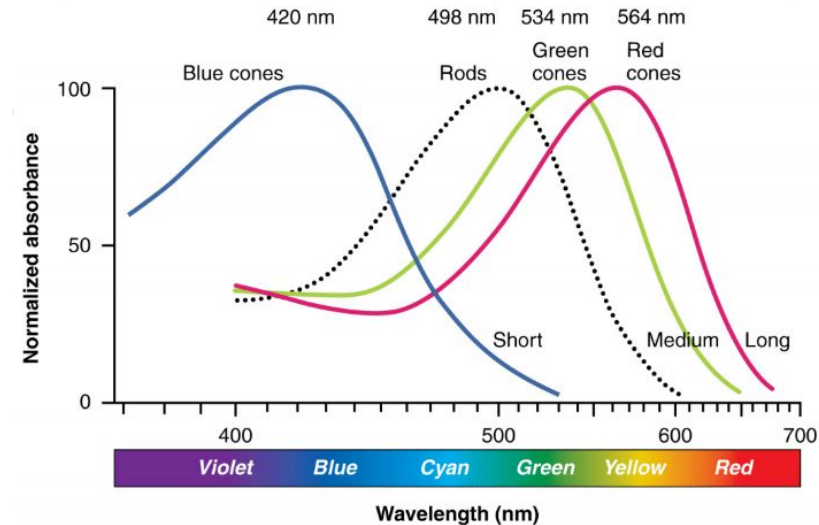
See references at the bottom of this page:

<https://medium.com/hipster-color-science/a-beginners-guide-to-colorimetry-401f1830b65a>

Talk by Dr. Michael Waters:

https://www.dropbox.com/s/7s9seplrwn3ea7p/Practical_Colorimetry_for_Scientific_Visualization_-_Michael_J_Waters_-_2021_3_14.pdf

Recommendation: Make your plots and graphics stand out for publication and presentation!



Examples

**What if we had tools to
prototype and make our
workflow a bit faster?**

Write a Python function that will fit a gaussian to a set of data points and give me the matplotlib plotting code



Certainly! Here's a Python function that fits a Gaussian curve to a set of data points using the SciPy library and provides the corresponding matplotlib plotting code:

python

Copy code

```
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

def fit_gaussian(x, y):
    # Define the Gaussian function
    def gaussian(x, amplitude, mean, stddev):
        return amplitude * np.exp(-((x - mean) / stddev)**2 / 2)

    # Perform the curve fit
    initial_guess = [np.max(y), np.mean(x), np.std(x)] # Initial guess for
    optimized_params, _ = curve_fit(gaussian, x, y, p0=initial_guess)

    # Generate data points from the fitted Gaussian function
    x_fit = np.linspace(np.min(x), np.max(x), 100)
    y_fit = gaussian(x_fit, *optimized_params)

    # Plot the original data points and the fitted Gaussian curve
    plt.plot(x, y, 'bo', label='Data')
    plt.plot(x_fit, y_fit, 'r-', label='Gaussian Fit')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.show()
```

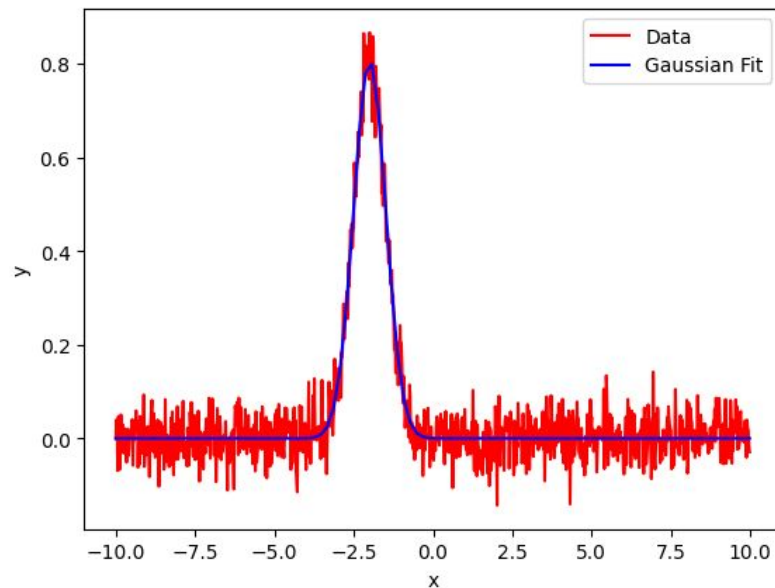
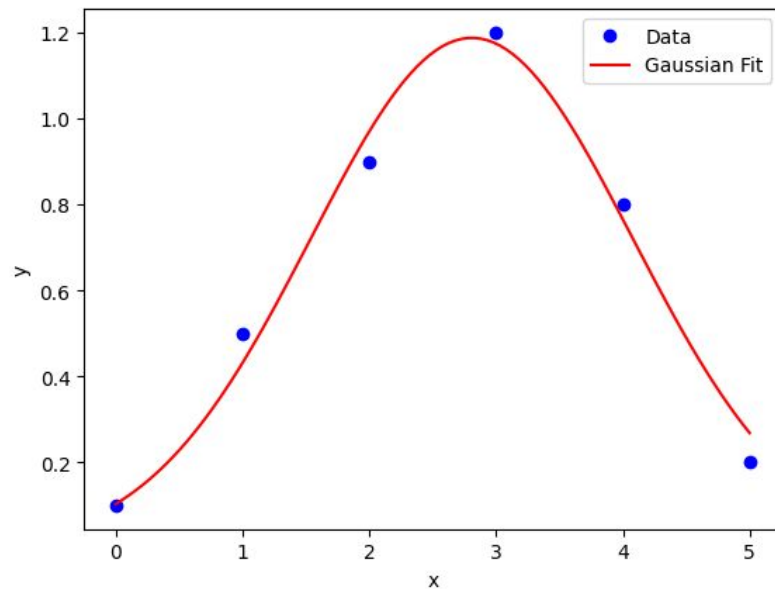
You can use this function by passing your data points as 'x' and 'y' arguments. Here's an example usage:

python

Copy code

```
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0.1, 0.5, 0.9, 1.2, 0.8, 0.2])

fit_gaussian(x, y)
```



Make a cosmology plot in Python of the Angular diameter distance in units of Megaparsecs as a function of redshift using AstroPy



To create a cosmology plot of the Angular Diameter Distance as a function of redshift using AstroPy, you can follow the steps below:

1. Install the necessary libraries:

```
pip install astropy matplotlib
```

2. Import the required modules:

```
python

import numpy as np
import matplotlib.pyplot as plt
from astropy.cosmology import Planck18 as cosmo
```

3. Define the redshift range:

```
python

redshifts = np.linspace(0, 10, 100)
```

4. Calculate the Angular Diameter Distance for each redshift using the chosen cosmology:

```
python

angular_diameter_distances = cosmo.angular_diameter_distance(redshifts)
```

5. Plot the Angular Diameter Distance as a function of redshift:

```
python

plt.plot(redshifts, angular_diameter_distances / 1e6) # Dividing by 1e6
plt.xlabel('Redshift')
plt.ylabel('Angular Diameter Distance (Mpc)')
plt.title('Angular Diameter Distance vs Redshift')
plt.grid(True)
plt.show()
```

Putting it all together, here's the complete code:

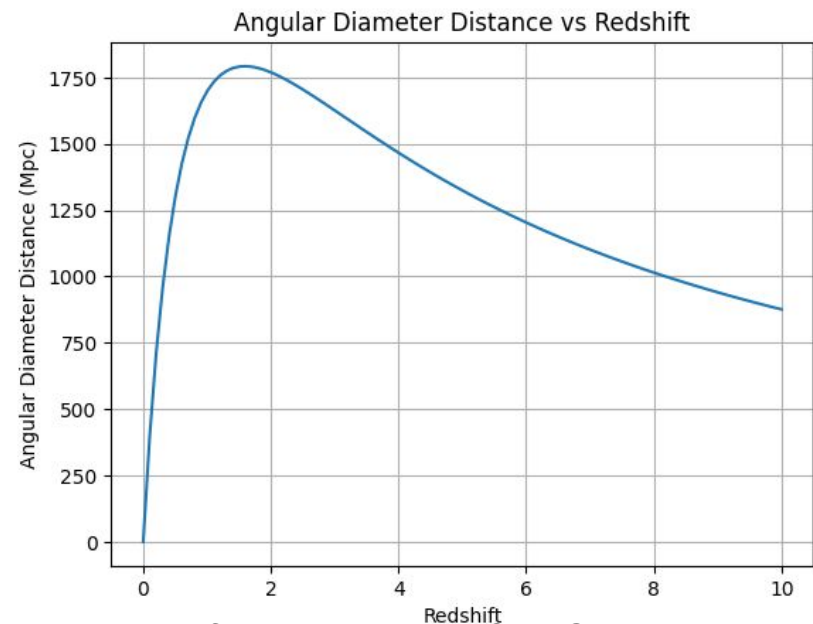
```
python

import numpy as np
import matplotlib.pyplot as plt
from astropy.cosmology import Planck18 as cosmo

redshifts = np.linspace(0, 10, 100)
angular_diameter_distances = cosmo.angular_diameter_distance(redshifts)

plt.plot(redshifts, angular_diameter_distances / 1e6) # Dividing by 1e6 to
plt.xlabel('Redshift')
plt.ylabel('Angular Diameter Distance (Mpc)')
plt.title('Angular Diameter Distance vs Redshift')
plt.grid(True)
plt.show()
```

This code will generate a plot showing the Angular Diameter Distance in units of Megaparsecs as a function of redshift, using the Planck 2018 cosmology. Feel free to adjust the redshift range or use a different cosmology as needed.



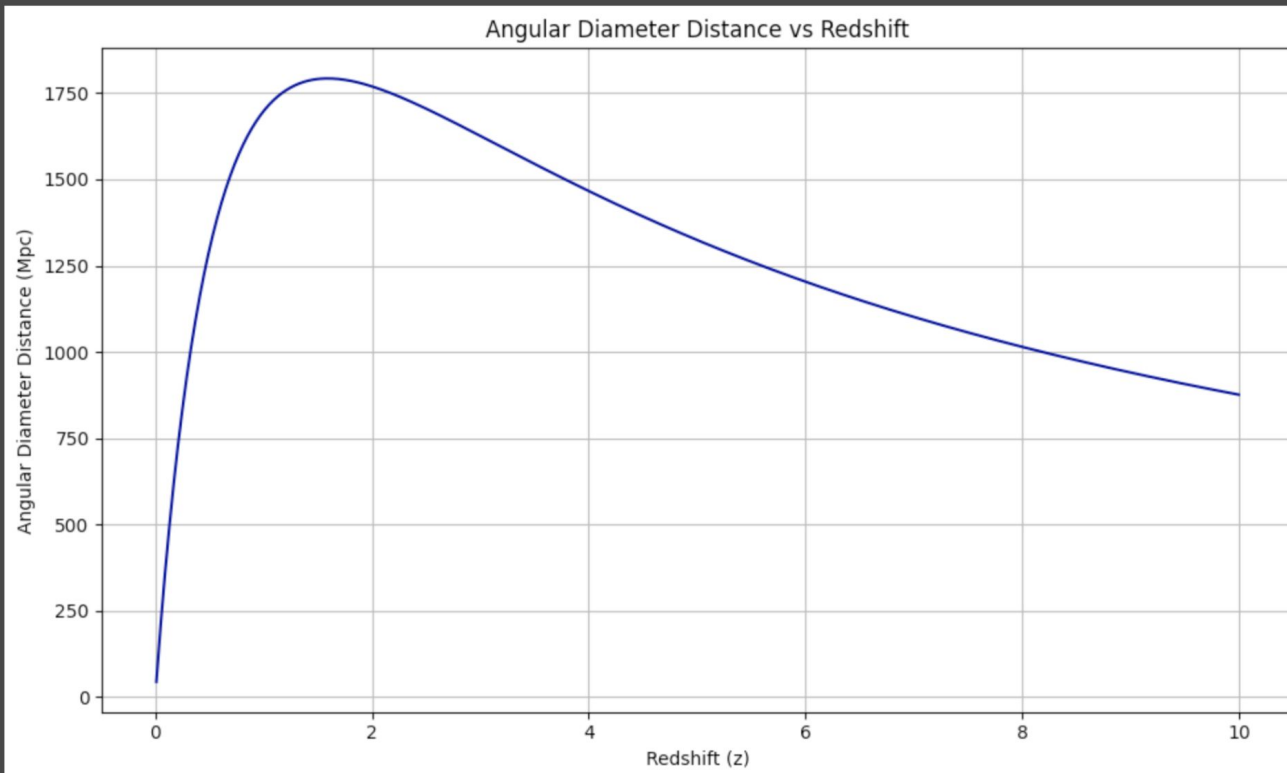
*Had to fix the units with ChatGPT...but it was close!


```
import numpy as np
import matplotlib.pyplot as plt
from astropy.cosmology import Planck18 as cosmo # You can choose different cosmologies
import astropy.units as u

# Define redshift range
z = np.linspace(0.01, 10, 500)

# Compute Angular Diameter Distance in Mpc
d_a = cosmo.angular_diameter_distance(z) # Returns Quantity in Mpc

# Plot
plt.figure(figsize=(10, 6))
plt.plot(z, d_a.value, color='darkblue')
plt.xlabel("Redshift (z)")
plt.ylabel("Angular Diameter Distance (Mpc)")
plt.title("Angular Diameter Distance vs Redshift")
plt.grid(True)
plt.tight_layout()
plt.show()
```



Python

```
import matplotlib.pyplot as plt
from astropy.cosmology import WMAP9 as cosmo # Use WMAP9 cosmology by default
import numpy as np

# Define redshift range
z_vals = np.linspace(0, 2, 100) # Redshift values from 0 to 2 with 100 points

# Calculate angular diameter distance
ang_diameter_dist = cosmo.angular_diameter_distance(z_vals)

# Plot the data
plt.figure(figsize=(8, 6))
plt.plot(z_vals, ang_diameter_dist.to(u.Mpc), label="Angular Diameter Distance (Mpc)")

# Set labels and title
plt.xlabel("Redshift (z)")
plt.ylabel("Angular Diameter Distance (Mpc)")
plt.title("Angular Diameter Distance vs. Redshift (WMAP9 Cosmology)")

# Add legend
plt.legend()

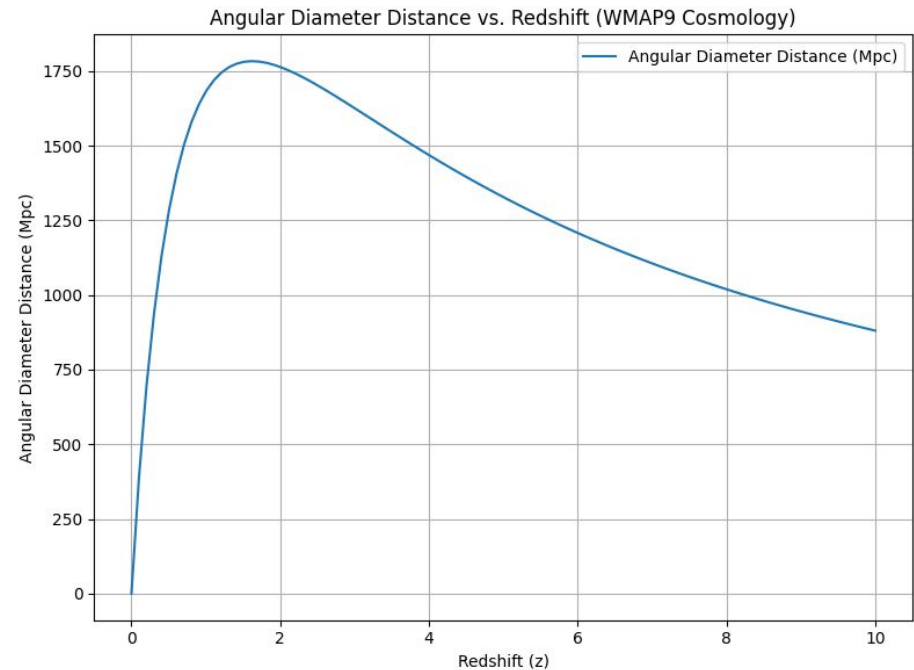
# Show the plot
plt.grid(True)
plt.tight_layout()
plt.show()
```

Use code [with caution](#).



This code does the following:

1. Imports necessary libraries: `matplotlib.pyplot` for plotting, `astropy.cosmology` for cosmological calculations, and `numpy` for numerical operations.
2. Defines a `WMAP9` cosmology object from `astropy.cosmology`. You can change this to another pre-defined cosmology or create a custom one.
3. Creates a list of redshift values (`z_vals`) ranging from 0 to 2 with 100 points.
4. Calculates the angular diameter distance for each redshift using `cosmo.angular_diameter_distance` and converts the units to Megaparsecs (Mpc) using `astropy.units`.
5. Creates a plot using `plt.plot`.



*Google Gemini was close - did not import the units from AstroPy, and only went to a redshift of 2.

BR

Write a Python function to download and display an optical image of the galaxy M51



To download and display an optical image of the galaxy M51, you can use the `'astroquery'` package to retrieve the image from an online database. Here's a Python function that accomplishes this:

python

Copy code

```
import matplotlib.pyplot as plt
from astropy.io import fits
from astroquery.skyview import SkyView

def download_and_display_m51_image():
    # Set the coordinates and image size for M51
    target_name = "M51"
    ra = 202.4697 # Right Ascension in degrees
    dec = 47.1953 # Declination in degrees
    image_size = 20 # Size of the image in arcminutes

    # Query the online database and download the optical image
    images = SkyView.get_images(position=f"{ra} {dec}", survey='DSS', coordi
    image = images[0]

    # Load the downloaded FITS file
    hdulist = fits.open(image[0])
    image_data = hdulist[0].data

    # Display the image
    plt.figure(figsize=(8, 8))
    plt.imshow(image_data, cmap='gray')
    plt.title(target_name)
    plt.axis('off')
    plt.show()
```

To use this function, you need to have the `'astroquery'` package installed. You can install it via `'pip'`:

Copy code

```
pip install astroquery
```

Once installed, you can call the `'download_and_display_m51_image()'` function to download and display the optical image of the galaxy M51. It will retrieve the image from the Digital Sky Survey (DSS) using the specified coordinates and image size. The resulting image will be displayed using Matplotlib.

python

Copy code

```
download_and_display_m51_image()
```

This will open a new window displaying the optical image of the galaxy M51. Adjust the target coordinates and image size as needed to retrieve images of other celestial objects.

There are some issues with this one - but they are fixable...

Query for an image...

```
!pip install astroquery

import matplotlib.pyplot as plt
from astropy.io import fits
from astroquery.skyview import SkyView

def download_and_display_m51_image():
    # Set the coordinates and image size for M51
    target_name = "M51"
    ra = 202.4697 # Right Ascension in degrees
    dec = 47.1953 # Declination in degrees
    image_size = 400 # Size of the image in pixels

    # Query the online database and download the optical image
    images = SkyView.get_images(position=f"{ra} {dec}",
                                survey='DSS',
                                coordinates='J2000', pixels=image_size)
    image = images[0]

    # Load the downloaded FITS file
    hdulist = image[0]
    image_data = hdulist.data

    # Display the image
    plt.figure(figsize=(8, 8))
    plt.imshow(image_data, cmap='gray')
    plt.title(target_name)
    plt.axis('off')
    plt.show()
```

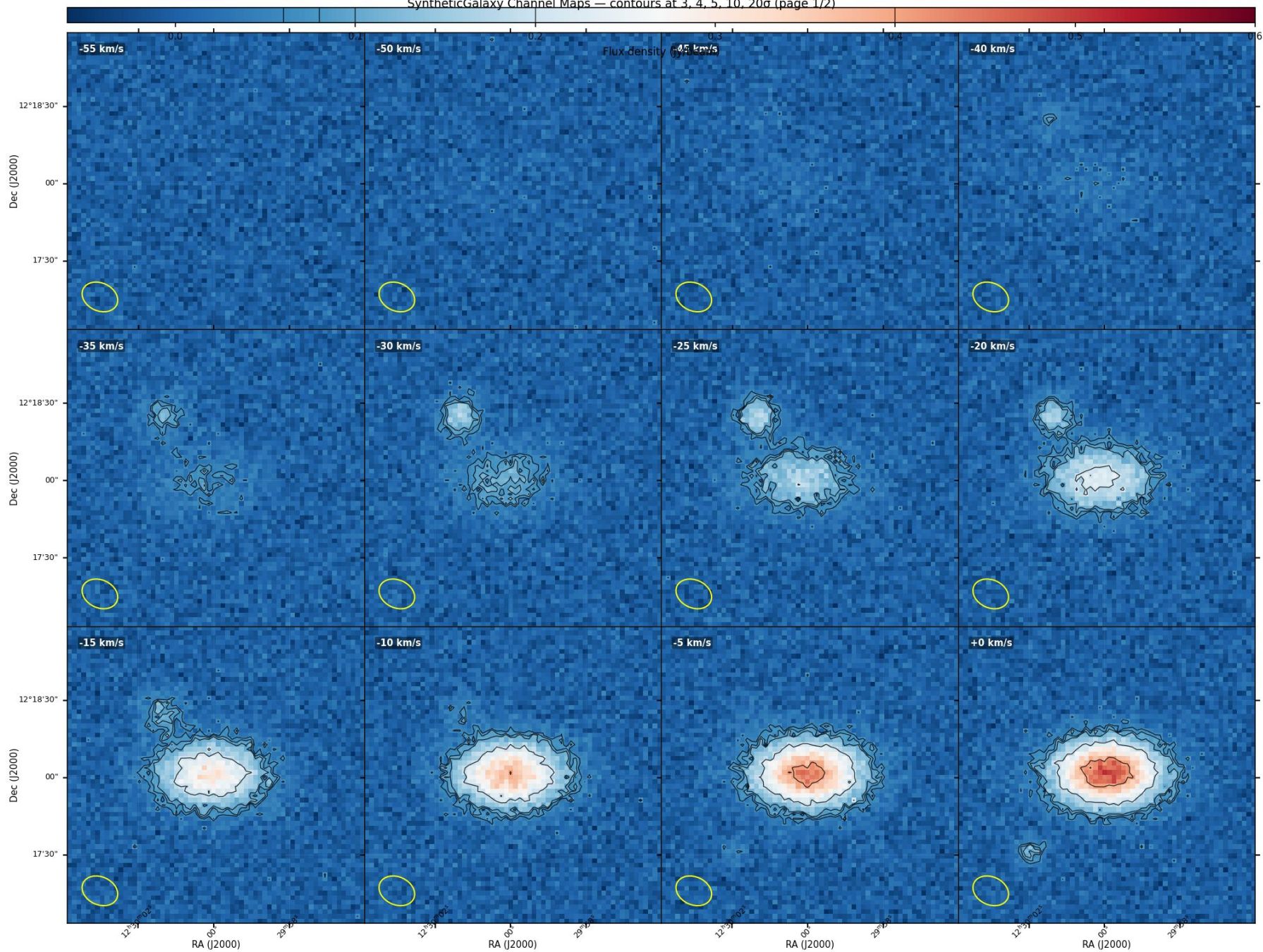
M51



Contour Maps with Claude

Make a Python script that will create channel maps for an astronomical data cube. The code should allow me to define a noise level, with contours at 3,4,5,10, and 20 sigma - but I should have a keyword parameter that will allow me to change that. There should be 12 maps on a page, labeled with declination on the y-axis, and right ascension on the x-axis. There should be no white space between the plots. The ellipse beam should be shown on each plot to scale. Generate some synthetic data to test out the code. Export a PDF and PNG file. Run the code and show me what the image would look like.

```
make_channel_maps(  
    cube, header,  
    rms=0.05,  
    sigma_levels=[3, 5, 10],  
    # ← change contour levels here  
    maps_per_page=12,  
    cols=4,  
    cmap="viridis",  
    output_pdf="my_cube.pdf",  
)
```

Prompts for Web Applications...

I have a large galaxy catalog in a CSV file, with entries for each galaxy. I also have a directory of approximately 31,000 individual formatted txt spectra files, each corresponding to an entry in the catalog.

The entries for the catalog have the header of:

AGCNR,Name,RAdeg_HI,DECdeg_HI,RAdeg_OC,DECdeg_OC,Vhelio,W50,sigW,W20,Hlflux,sigflux,SNR,RMS,Dist,sigDist,logMH,siglogMH,Hlcode

Each formatted txt spectra file has five columns:

Heliocentric velocity [km/s]

Radio frequency [MHz]

Flux Density [mJy]

Subtracted polynomial baseline fit [mJy]

Normalized spectral weight

These columns in each spectra file start on the 41st line of each spectra txt file.

I would like to create a self contained interactive web app with CSS and javascript that can be hosted anywhere, or just run locally within a web browser with search capability such that I can have an view page that shows the galaxy spectra, all information on that galaxy based on the AGCNR, and will also retrieve image cutouts from other astronomical surveys. The app should indexed by AGCNR. The spectra files follow the filename pattern of Axxxxxx.txt, with leading zeroes if needed. You need to include plotting code that will generate plots of the spectra in real time on the fly to include for each galaxy's page. The plot can use any javascript or CSS tool, and should plot the heliocentric velocity column on the X-axis and the flux density on the Y-axis.

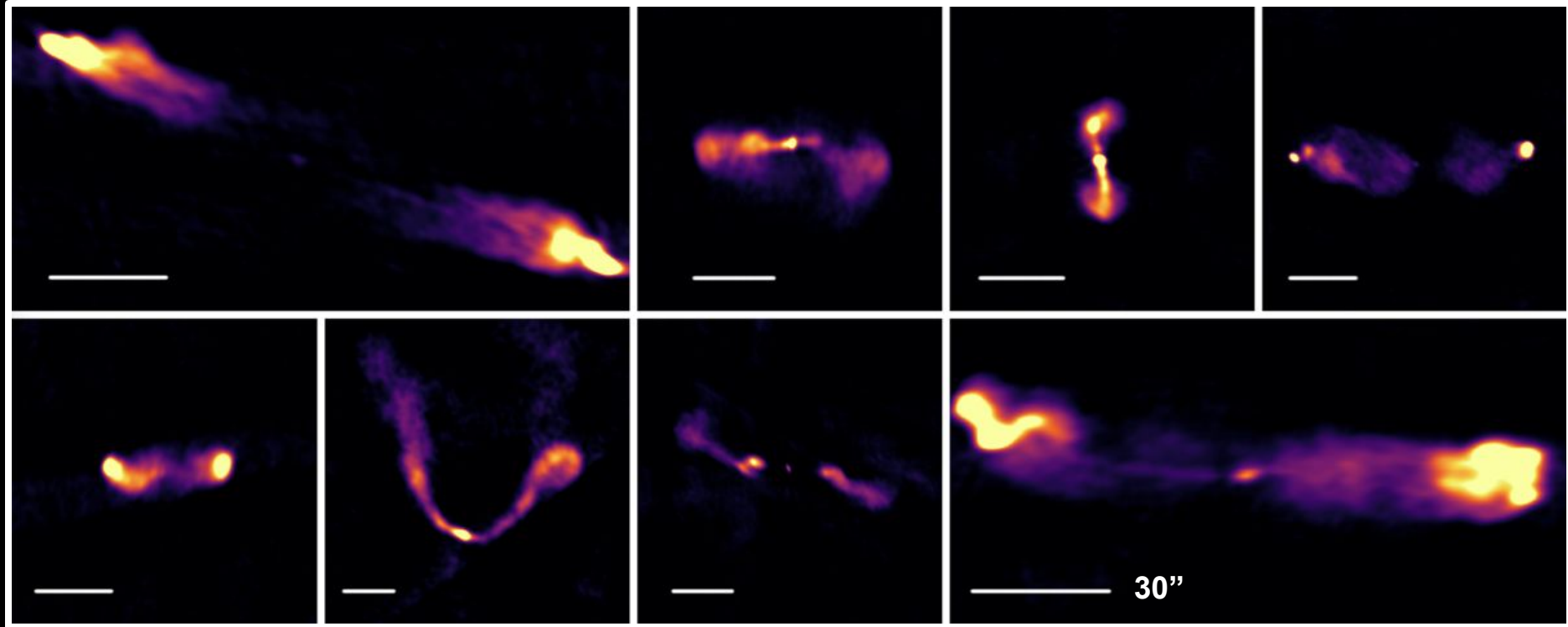
Design the web page with with an astronomical observatory terminal aesthetic.

Go through the links on the left hand side of the NRAO Science Program Archives, listed as a Year and a Letter, for example: 2024A. The Letter indicates the semester. On each page there is a table of accepted proposals, with the columns of the Principal Investigator (PI), the ID of the proposal, the title of the proposal, the hours awarded, and Type of the Proposal. The website can be found here:

<https://science.nrao.edu/science/science-program>

Catalog all this information, and create a web app that can be run locally where I can query this information to determine what subjects are most popular, and where success might be found in proposing for NRAO telescopes - what gaps are there in the science?

TensorFlow Example: VLASS Galaxy Classification



<https://github.com/brkent/SummerStudent2026>

See the paper by Lacy et al. 2020 <https://ui.adsabs.harvard.edu/abs/2020PASP..132c5001L/abstract>
Over 600 citations so far. 2641 refereed authors and counting...

Machine Learning/Deep Learning Neural Networks/Artificial Intelligence

NVidia Deep Learning:

<https://www.nvidia.com/en-us/deep-learning-ai/education/>

PyTorch (Meta/Facebook):

<https://pytorch.org/>

Tensorflow (Google):

<https://www.tensorflow.org/>



Check out the NVidia GTC: <https://www.nvidia.com/en-us/gtc/>

Recommendation: *Please subscribe and sign up for these free tutorials and resources!*